



Hot Chips Tutorial, Part-I: Update on RISC-V standards and adoption, including profiles and platforms

Krste Asanovic

Chief Architect, RISC-V International

Chief Architect, SiFive

Professor Emeritus, EECS, UC Berkeley

Stanford, CA

August 23, 2026



What is RISC-V?

- A *simple* ISA designed for teaching?
- A *flexible* ISA with hundreds of extensions that can be arbitrarily mixed to enable highly specialized cores?
- A *tightly controlled* ISA providing binary-compatibility for rich software ecosystems across many CPU vendors?

All of the above!



Talk Outline

- RISC-V History and Background
- RISC-V Specification Structure and Status
 - Bases
 - Extensions
 - Privileged Architecture
 - Profiles
 - Subsystems and Platforms
- RISC-V FAQs, Myths, and Fallacies



RISC-V is 16 Years Old!

RISC-V Official Birthday, May 18, 2010

Abbreviated RISC-V history:

2010-2014 Initial development at UC Berkeley and engagement with broader community

Hot Chips Roll Out, 2014

2015-2019 RISC-V Foundation founded, base specs ratified, RV64GC, first commercial offerings

First Hot Chips Tutorial, 2019

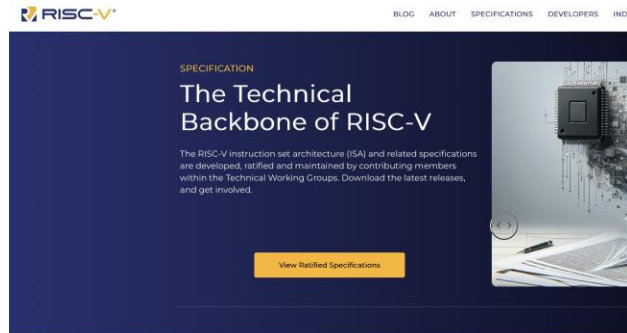
2020-2025 Widespread commercial embedded deployments, filling out standards, RVA23 ISA profile, build out of foundational software

2026- Push into server processors, first RVA23 silicon available

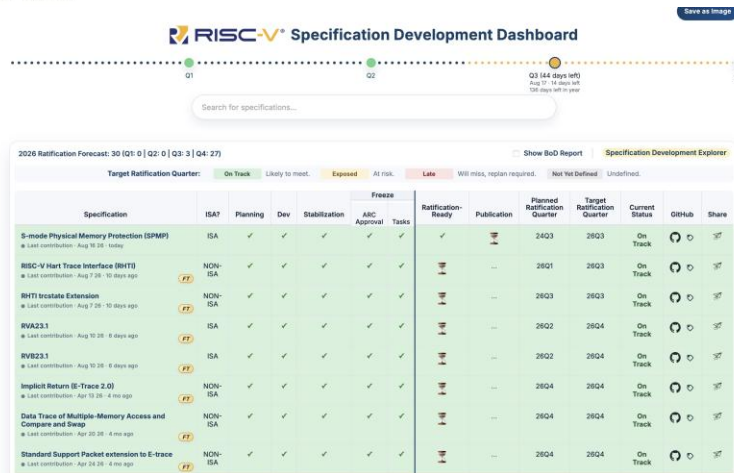
Second Hot Chips Tutorial, 2026



Hot Chips 2014



News and Insights



- Non-Profit Standards Organization, based in Switzerland
- 23 Premier Board/TSC Members, 134 Strategic Members, and Academic, Community, and Individual Members
- RISC-V International approved as PAS (*publicly available specification*) Submitter by ISO/IEC Joint Technical Committee



RISC-V is Different

- A ***global open-standard ISA*** developed by the community
 - Not tied to business fortunes of any owner
- ***Vastly more processor implementations*** than any ISA in history
 - Commercially licensable IP cores
 - In-house-designed proprietary cores
 - Open-source cores (SERV is just 125 FPGA LUTS, or 2.1K ASIC gates)
- Targets ***all processor sockets***
 - Not just embedded
 - Not just mobile
 - Not just servers
 - Not just AI/HPC
 - Not just automotive
 - Not just secure cores
 - Not just DSPs
 - Not just FPGA soft cores
 - Not just Ethereum virtual machine



Understanding RISC-V Specs

To support the vast RISC-V Universe, ISA spec is highly modular, factoring out common pieces to allow reuse of specifications and software support.

- **Base ISAs**

- Define fundamental parameters and operation of each RISC-V ISA (address space width, number of integer registers, endianness, ...) and how these affect extensions

- **ISA Extensions**

- Add features (compressed instructions, floating-point, vectors, hypervisor, ...) to a base ISA
- Shared across base ISAs

- **ISA Profiles**

- Define ISA base and mandatory extensions for given vertical market, together with selected profile options

- **Platforms and Subsystems**

- Beyond ISA, defining whole hardware platform standards



RISC-V Base ISAs

- A base ISA must be a complete unprivileged ISA
 - Informally, forms complete target for a C compiler
- Four ratified bases: **RV32I**, **RV64I**, **RV32E**, **RV64E**
 - 32b or 64b address space and integer register width (XLEN parameter)
 - 32 (I) or 16 (E) integer registers
- Further bases under development
 - **RV64Y** CHERI (adds hardware capabilities)
 - **RV128** variants for 128b address space (early proposals developing)
- *Cleanup of base specification structure in progress*
 - *Big-endian as different base, not option on base*
 - *ZFinX (Floating-point out of integer registers) being recast as new base not an extension*



RV32I Base Unprivileged Instructions

32b program counter **pc**

31 x 32b integer registers,
x1-x31
plus zero register **x0**

Just another “boring”
RISC architecture.

imm[31:12]				rd	0110111	LUI	
imm[31:12]				rd	0010111	AUIPC	
imm[20 10:1 11 19:12]				rd	1101111	JAL	
imm[11:0]			rs1	000	rd	1100111	JALR
imm[12 10:5]		rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]		rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]		rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]		rs2	rs1	101	imm[4:1 11]	1100011	BGE
imm[12 10:5]		rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]		rs2	rs1	111	imm[4:1 11]	1100011	BGEU
imm[11:0]			rs1	000	rd	0000011	LB
imm[11:0]			rs1	001	rd	0000011	LH
imm[11:0]			rs1	010	rd	0000011	LW
imm[11:0]			rs1	100	rd	0000011	LBU
imm[11:0]			rs1	101	rd	0000011	LHU
imm[11:5]		rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]		rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]		rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]			rs1	000	rd	0010011	ADDI
imm[11:0]			rs1	010	rd	0010011	SLTI
imm[11:0]			rs1	011	rd	0010011	SLTIU
imm[11:0]			rs1	100	rd	0010011	XORI
imm[11:0]			rs1	110	rd	0010011	ORI
imm[11:0]			rs1	111	rd	0010011	ANDI
0000000		shamt	rs1	001	rd	0010011	SLLI
0000000		shamt	rs1	101	rd	0010011	SRLI
0100000		shamt	rs1	101	rd	0010011	SRAI
0000000		rs2	rs1	000	rd	0110011	ADD
0100000		rs2	rs1	000	rd	0110011	SUB
0000000		rs2	rs1	001	rd	0110011	SLL
0000000		rs2	rs1	010	rd	0110011	SLT
0000000		rs2	rs1	011	rd	0110011	SLTU
0000000		rs2	rs1	100	rd	0110011	XOR
0000000		rs2	rs1	101	rd	0110011	SRL
0100000		rs2	rs1	101	rd	0110011	SRA
0000000		rs2	rs1	110	rd	0110011	OR
0000000		rs2	rs1	111	rd	0110011	AND
fm	pred	succ	rs1	000	rd	0001111	FENCE
0000000000000			00000	000	00000	1110011	ECALL
0000000000001			00000	000	00000	1110011	EBREAK

ISA Extensions

- Initial spec had extensions with single-letter names:
 - **M** – multiply/divide, **A** – atomics, **F** – single-precision floats, **D** – double-precision, **Q** – quad-precision, **C** – compressed, **V** – vector, etc.
 - (**G** introduced as alias for common set **MAFD**)
- Later expanded with **Z*** names for unprivileged extensions, with first letter after **Z** giving category
 - **Zi*** - integer extensions, **Za*** - atomics, **Zv*** - vector extensions, etc.
- Extension's behavior depends on base
 - E.g., original **D** extension only has integer-float register move when **XLEN=64** (**zfa** extension adds these for **XLEN=32**)
 - Allows reuse of same spec across multiple bases, e.g., **CHERI** will use same FP specs as other RISC-V bases



RISC-V Encoding Terminology

Standard: Defined by RVIA

Reserved: RVIA will eventually use for future standard extensions

Custom: Space for vendor-specific features, never used by RVIA

- ISA string convention **X<vendor>***, e.g., **Xsfvqdotq**

Non-Standard: Not defined by RVIA

Non-Conforming: Non-standard feature in standard or reserved encoding space

Harts and Execution Environments

ISA specifications describe abstracted behavior of RISC-V *harts* inside an *execution environment*

- *Hart* (**HAR**dware **T**hread) is abstraction of hardware processor
 - Hart always “beating”, autonomously running instructions within an execution environment
- *Execution environment* defines initial program state, how harts are created/appear, properties of memory regions, I/O, handling of exceptions and interrupts, environment calls, etc.
 - The Linux user ABI is an example of an execution-environment specification

IMPORTANT: RISC-V ISA specs are not (only) for describing hardware implementations



Privileged Architecture

RISC-V Privilege modes:

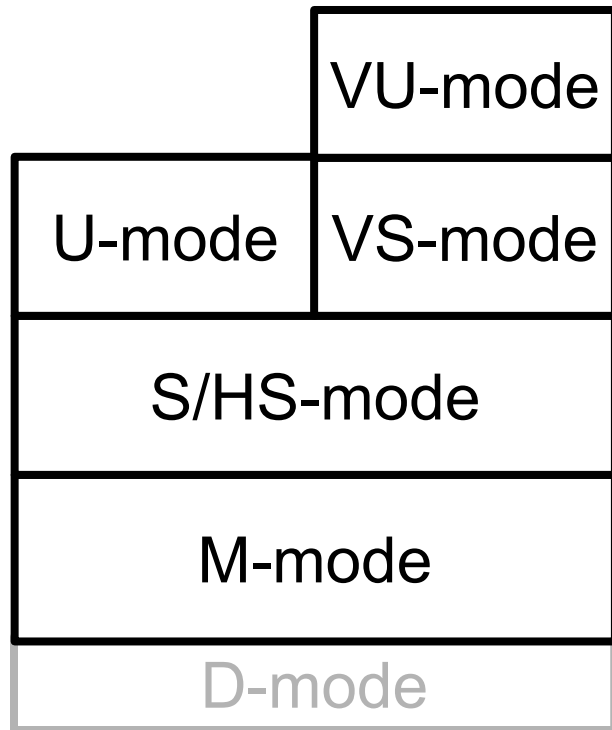
- User (U-mode)
- Supervisor (S-mode)
- Machine (M-mode)

Hypervisor extension (“**H**”) enhances S-mode to HS-mode and adds:

- Virtual User (VU-mode)
- Virtual Supervisor (VS-mode)

with 2-stage address translation (same page-table format in both stages, and for standard IOMMU)

Also, debug mode (D-mode) used by hardware debug module





Privilege Layering and Extensions

RISC-V ISA enforces clean layering of privilege modes

- ISA specs do not make assumptions about existence of higher-privilege modes

RISC-V ISA has no virtualization holes

- e.g., a hypervisor can run in user-mode with execution environment using traps to emulate all privileged features

Privileged features are separated into extensions visible at each privilege mode, and extension names begin with “s” followed by mode:

- **S_m*** - machine-mode extensions
- **S_h*** - hypervisor-mode extensions
- **S_s*** - supervisor-mode extensions
- **S_v*** - virtual-memory-related extensions
- **S_d*** - debug-mode extensions

ISA Strings

- Constructed with leading base name followed by extensions:
 - **RV64IMAFDC**
 - Case-insensitive: `rv64imafdc` = `RV64IMAFDC`
- Can get long if expanded fully, e.g.:

```
rv64imafdc_b_v_h_zic64b_zicbom_zicbop_zicboz_ziccamoa_ziccif_zicclsm_
ziccrse_zicfilp_zicfiss_zicntr_zicond_zicsr_zifencei_zihintntnl_zihint
pause_zihpm_zimop_sdext_sdtrig_shcounterenw_shgatpa_shtvala_shvsatpa_
shvstvala_shvstvecd_smaia_smcsrind_smctr_smpm1_smrnmi_smstateen_sslp
13_ssaia_sscpctr_sscfpmf_sscounterenw_sscsrind_ssctr_ssnpm1_ss qosid1
_ssstateen_sstc_sstvala_sstvecd_ssu64xl_svade_svbare_svinval_svnapot_
svpbmt_svukte_za64rs_zamal6b_zawrs_zfa_zfbfmin_zfh_zca_zcb_zcd_zcmop_
zba_zbb_zbs_zkr_zkt_zvbb_zvbc_zvfbfmin_zvfbfwma_zvfh_zvkng_zvksg_zvkt
_zvl128b
```

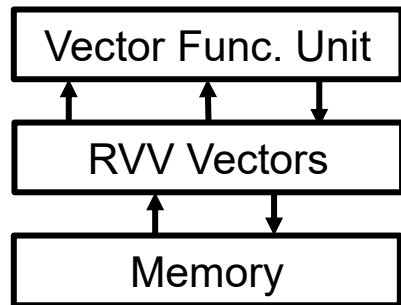
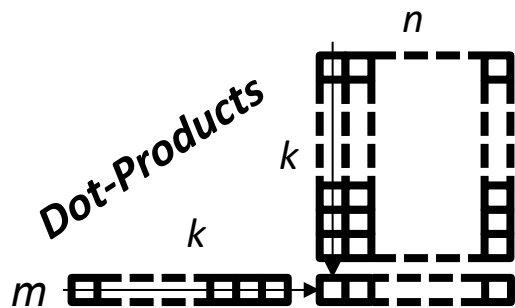


RISC-V Vector (RVV) Extension

- Adds 32 vector registers with implementation-dependent length (VLEN bits)
 - Ratified November 2021
 - Many implementations from VLEN=32 to VLEN=16,384, with up to 2048b in vector datapath width (DLEN)
- LMUL feature gangs vector registers to support efficient mixed-width computations and to reduce instruction bandwidth demands
- Vector ISA supports vector-length agnostic (VLA) code
 - Binary code following VLA discipline can run on different VLEN, taking advantage of full vector length without recompilation
 - Same binary code runs on everything from narrow 32b-wide datapaths on embedded microcontrollers to 2048b-wide datapaths on AI accelerators

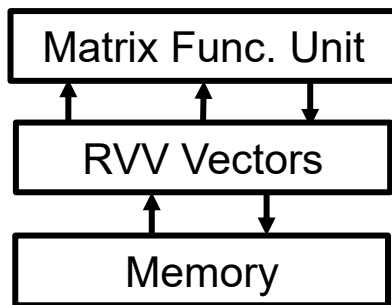
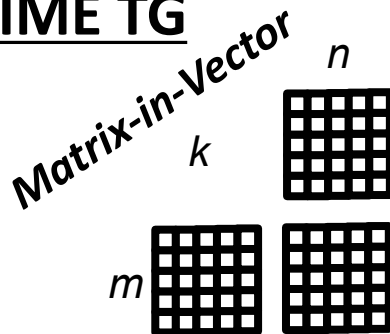
RISC-V Matrix Extensions, 1+3 Approaches

Zvbdota Fast-Track

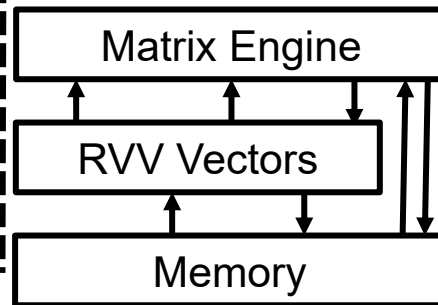
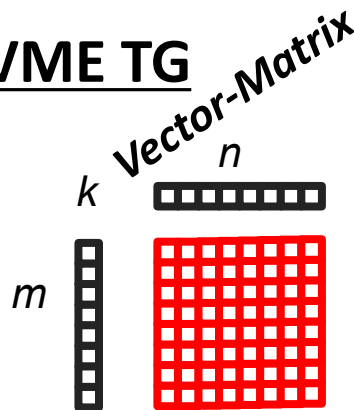


- ☐ Element in existing RVV vector register
- ☐ Element in added matrix registers

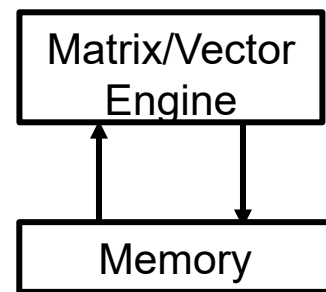
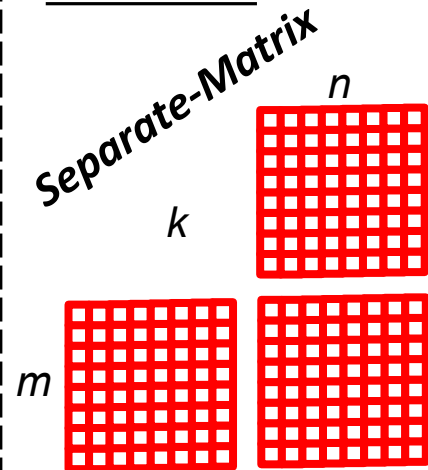
IME TG



VME TG

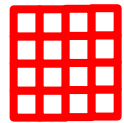


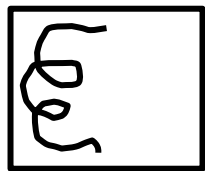
AME TG



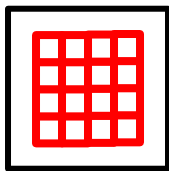
AI Compute Organizations

 General-purpose
compute

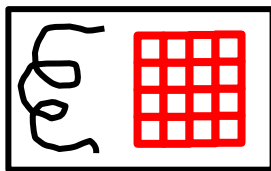
 Number-crunching
compute



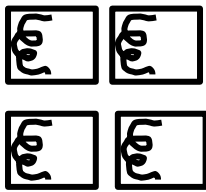
CPU



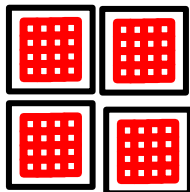
GPU



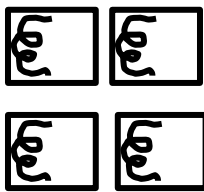
RISC-V scalar/vector/matrix core



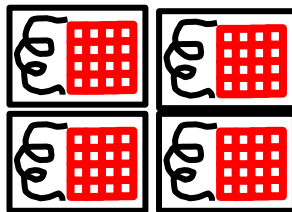
Host



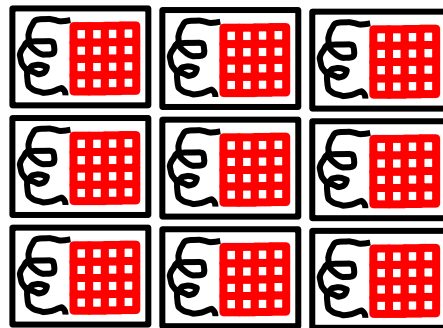
Device



Host



Device



Self-Hosted

Wheel of Reincarnation

Myer and Sutherland, "On the Design of Display Processors", 1968



Why RISC-V is unique for AI

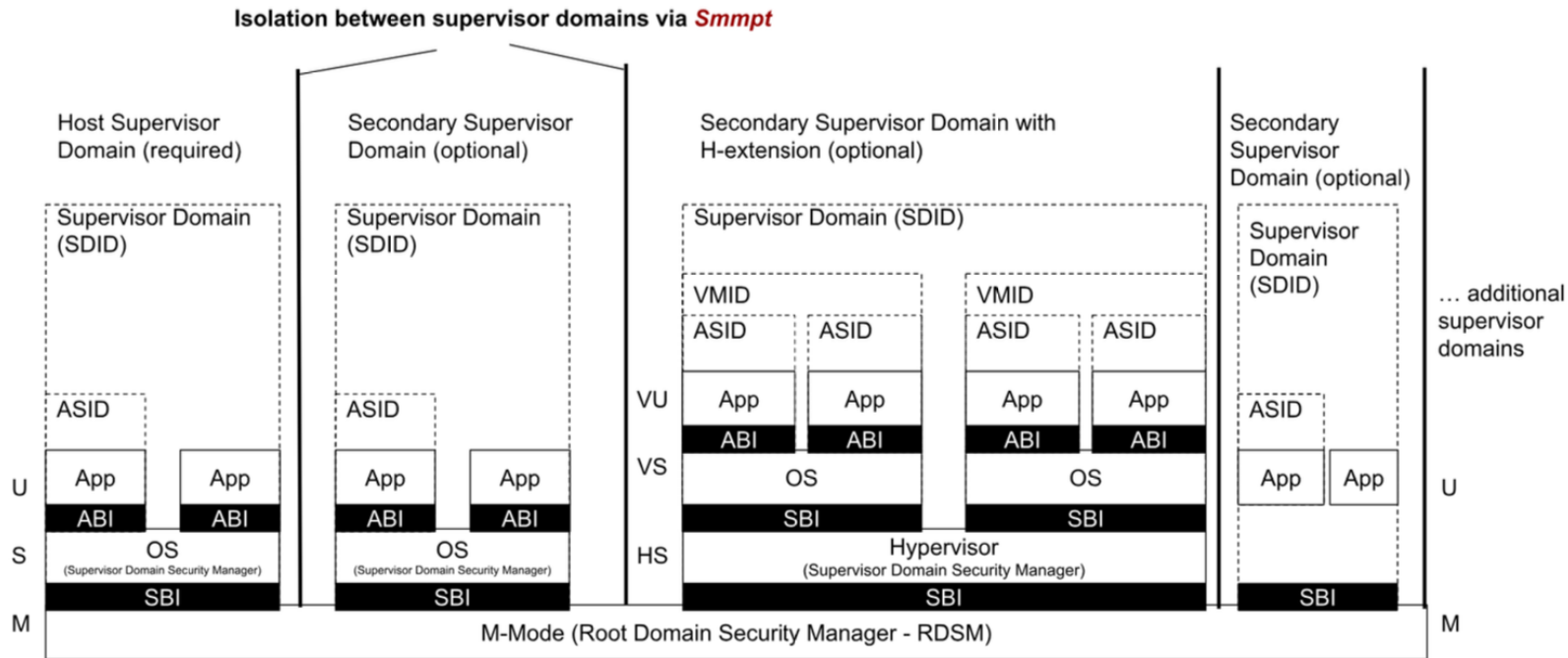
- Among competing ISAs, only RISC-V works well for:
 - AI Host ISA (e.g., Nvidia port of CUDA stack to RISC-V)
 - AI Device ISA (e.g., many AI accelerators based on RISC-V)
 - Self-hosted AI ISA (e.g. edge AI, plus upcoming agentic CPUs)
- RISC-V ISA also scales down to MMU-less embedded devices while keeping same vector and matrix extensions
- RISC-V supports AI innovation and differentiation through custom extensions
- Share standard compute kernel and software stack costs across all possible AI compute environments

Low-Level Security Features

Support for TEEs, trusted boot code and trusted firmware:

- **PMP** (Physical Memory Protection) registers allow M-mode to control access from lower-privilege modes to physical memory
 - **Sspmp** extension adds second level of **supervisor PMP** registers, for nested memory protection
- **RISC-V Worlds** provide bus-level protection of SoC components, with multiple supported Worlds (up to XLEN Worlds)
 - All bus transactions are tagged with World ID, supporting target-side checking of access permissions
 - Security is not a binary property! RISC-V Worlds provides strong isolation between multiple mutually distrustful compartments.

Supervisor Domains for Confidential Compute



- Memory-resident page-based memory-protection table allows M-mode to isolate multiple supervisor/hypervisor stacks
- IOMMU supports supervisor domains with same page-table format

Other ISA Security Features

- Extensive scalar and vector crypto support
- Data-independent execution time features
- Control-Flow Integrity checks (landing pads, shadow stack)
- Pointer masking for KASAN/HWASAN
- Constant-time TLB faults to privileged memory
- Unexpected trap handling
- CHERI (full-blown hardware capability system, in development)
- Speculation Barriers (in development)
- Temporal fences (in development)
- Lightweight memory tagging (in development)
- Memory encryption contexts (in development)
- + Many others in discussion and development



RVA Profiles for Binary Software Ecosystems

- Binary software ecosystems want to run on largest number of devices, so target “lowest common denominator” of fielded systems
- Hardware vendors don’t want to add features that won’t be used
- To remain competitive, architectures must support a growing set of features over time
- RISC-V RVA ISA **profiles** provide solution to this dilemma
 - An agreed roadmap of mandatory ISA features for hardware implementers to provide in each generation, so software can rely on widespread availability and exploit these features

RVA: Application-Processor Profiles

- RVA supports rich application-processor binary software ecosystems
 - Examples including Android, Linux distros, ...
 - RVA goal that new versions can run all software that ran on old versions
 - While many RVA sockets require the highest-performance cores (datacenter, laptop, mobile), others require low-cost cores with full feature compatibility (consumer devices)
- Ratified RVA profile specifications
 - RVA20 (retrospectively captures de-facto c.2016 standard "RV64GC")
 - RVA22 (significant clean up and multiple advances over RVA20)
 - **RVA23 profile (ratified October 2024), key features:**
 - Mandatory vectors
 - Mandatory hypervisor
 - High-performance vector crypto as localized option

Profiles: Mandates + Options

RVA profiles contain mandatory extensions plus four kinds of options:

- **Localized options:** required for different jurisdictions (e.g., crypto)
- **Development options:** new extensions likely to be made mandatory but in early part of lifecycle
- **Expansion options:** large implementation overhead and not always needed, but can be handled via runtime discovery (e.g., matrix engine)
- **Transitory options:** not clear if will remain in profile or be dropped (e.g., scalar crypto)



Profile Options

Localized and **Expansion** options are carefully curated

- These are longer-lived as options
- Coarse-grain to minimize software support cost
 - Many fewer profile options than ISA extensions, not every extension is a profile option

Development options can be finer-grained, as will become mandatory and no longer listed separately

Major versus Minor RVA Releases

- **Major** releases signify substantial new mandatory functionality
 - RVA20 first major release
 - RVA23 second major release (vector, hypervisor mandatory)
 - Multiple years between major releases
- Future **minor** releases only introduce options, no mandates
 - Ensures new standard software continues to run on hardware from previous major release
 - While enabling hardware and software support of new features (e.g., development options that will become mandates)
 - Regular cadence of minor releases to track new extension ratifications
 - First RVA minor release RVA23.1 (**rva23p1**) close to ratification

RVB23 Profile also Ratified

Not an “alternative” RVA – serves very different purpose:

- **RVB23** profile provides more flexibility for application processors used with custom software builds (e.g., Yocto/OpenEmbedded), not binary software distributions

Future RVB profiles will evolve independently from RVA

- As fewer mandates, likely fewer major releases

Optimization Guidance Options

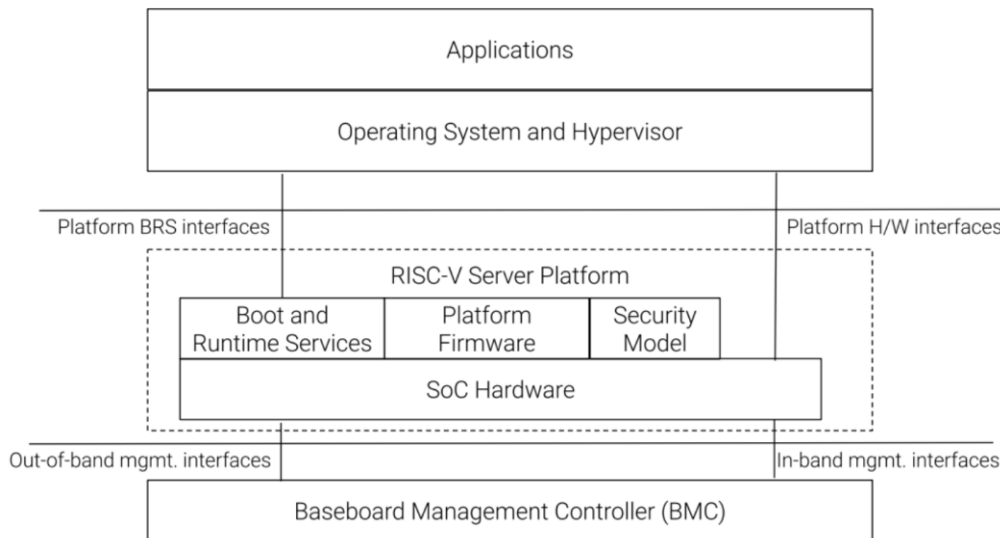
- Current ISA strings encode functionality, not performance
 - e.g., Zilsm says misaligned will work, but not how fast.
 - Some hardware vendors might believe trap-and-emulate is acceptable
 - Software pessimistically codes around misaligned accesses, even though high-performance cores have fast hardware support.
- New optimization guidance options align hardware and software to raise the bar for RISC-V application processors
 - Intended to be mandatory for future RVA profiles, but will first appear as development options to guide the community forward

Initial Optimization Guidance Options

Oilsm: Software should assume misaligned scalar load/store accesses and misaligned element accesses within vector load/store instructions will be handled more efficiently than alternative code sequences that avoid the misalignment.

Ovlt: Software should assume microarchitectures will optimize execution of vector instructions based on the dynamic value in the "vl" register, i.e., execution time of a vector instruction will not be improved by reducing LMUL (and hence VLMAX) based on statically known or dynamically determined application vector lengths.

RISC-V Server Platform Spec



- Goals are to ensure compatibility between RISC-V vendors, and maximize reuse of industry server standards adopted for other ISAs
 - RVA23-compliant Processors
 - Standard RISC-V AIA interrupts
 - PCIe/IOMMU
 - UART
 - UEFI/ACPI
 - TPM
 - BMC via MCTP/PLDM/Redfish/IPMI
 - Etc.

RVM Profile: Standardizing Microcontrollers

- While many microcontroller use cases benefit from extreme specialization, richer embedded software stacks (e.g. automotive) would benefit from standard profiles
- Richer microcontroller standard including:
 - RVM ISA profile mandating many extensions
 - RVM Subsystem standard including:
 - Standard fast interrupts
 - Standard nested protection (sPMP)
 - Standard security features (RISC-V Worlds)
 - Standard debug /trace support

RISC-V FAQs, Myths and Fallacies



RISC-V is an Open-Source Processor!

NO!!!!

RISC-V is an Open ISA Standard!

RISC-V is fragmented!

Fragmentation versus Diversity



Fragmentation:

Same thing done different ways



Diversity:

Solving different problems





RISC-V Reduces Fragmentation!

- x86 has only two vendors, but different hypervisors and IOMMU specs
- Multiple RISC-V vendors work together to define RVA profiles to align application processor hardware and software ecosystems
- ARM has four different vector architectures (VFP, NEON, SVE, MVE)
- Same RISC-V vector extension used in 64b apps cores, AI accelerators, and 32b embedded cores
- ARM has had multiple different microcontroller ISAs (ARM32, Thumb1, Thumb2) and limited 64b embedded offering with no code compression
- RISC-V supports 32b and 64b embedded with code compression
- RISC-V avoids SoC Balkanization! Can use same ISA for every socket in an SoC – no need to create proprietary controller ISA for every IP block.



RISC-V has too many extensions!

- RISC-V has ~200 extensions
- AArch64 has ~400
- x86_64 has ~800
- New extensions are a fact of life in a living, vibrant ISA, especially one supporting every type of processor
- RVIA community carefully manages extensions and makes them reusable across many types of processor



RISC-V is just cheaper!

- RISC-V enables competition
 - Yes, cheaper can be one consequence of competition, removing monopoly tax (lower cost is not a bad thing)
 - But competition also drives other metrics: performance, power, features
- RISC-V provides freedom to innovate but with support from a standard software stack
 - Not possible with legacy ISAs

Variable-Length Instructions Hurt!

- Variable-length only a minor annoyance in front-end designs
 - RISC-V encodes size in 2 bits, much simpler than x86
 - Advanced decoupled front-end designs don't need instruction bits to predict control flow
 - At low-end, even old 8b micros coped with variable-length fetch
 - Even Seymour Cray used variable-length instructions!
- Variable-length improves efficiency through static/dynamic code compression and/or more expressive instruction encodings
- Fixed-size instructions are either too big, or you run out of them
 - ARM and POWER have adopted prefix instructions to handle running out of opcode space – uglier than planning expansion from beginning



RISC-V requires Fusion to be Competitive!

No, opposite is true:

- x86 and ARM need fusion for compare+branch
- Even base RISC-V has single instruction for cmp+br

Fusion is optional on performant RISC-V cores

“Should we add more complex addressing modes to avoid fusion?”

- Simpler RISC-V instructions can be *optionally* fused, but complex macroinstructions would *mandate* cracking into microops.
- In any case, complex addressing modes have limited measured improvements, especially on wider superscalars, and can have significant PPA impact



Bonus Slide: How to write a “RISC-V Sucks” blog post

- Pick a complex instruction in your favorite architecture that RISC-V lacks. Show that RISC-V needs three instructions to do the same. Make sure to ignore microarchitecture PPA impact, or importance in real code, as that would reduce the impact of your insightful analysis.
- Pick a RISC-V system designed for application X and compare with your favorite system designed for and running application Y, as every system must clearly be good at running all applications.
- Assume that your use case is the only one in the universe and bemoan lack of specialized features you need, also make sure to disparage any RISC-V feature or optionality that doesn't fit your use case.
- Make sure to pick a poor implementation or one in an older process technology to demonstrate RISC-V cannot provide high performance, as it is clearly impossible to improve performance with more design investment or better process technology.



RISC-V is not High Performance!

- No, there is no technical barrier to efficient high-performance RISC-V implementations, and many technical advantages
- Main barrier was size of investment in design and fabrication needed versus current high-end market demand, but flywheel has started spinning as high-end RISC-V ecosystem is growing
- This year, first RVA23 servers are appearing from multiple vendors
- Stay tuned!